

MEDIATEK

MT8676 Hypervisor Secure Boot User Manual

Version: 1.0
Publication date: 2024-11-14

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-11-14	Peak	Official version

Table of Contents

Version History2

Table of Contents3

List of Figures3

List of Tables3

1 Secure Boot4

1.1 Overview 4

1.1.1 Abbreviations 4

1.1.2 BootRom 4

1.2 Secure Boot 5

1.2.1 Secure Boot Check Process 5

1.3 Download Agent Authentication (DAA) 7

1.4 Security Feature Configuration 7

1.4.1 Generate Key Pairs. 7

1.4.2 Enable Secure Boot. 9

1.4.3 Compilation Software 10

1.4.4 Signature First-Loader 10

1.4.5 Signature FIT Format Image 11

1.4.6 Sign the Yocto File System Image. 11

1.4.7 Sign the MTK Android Format Image. 11

1.5 Signature DA..... 12

1.6 Generate Authfile..... 12

Exhibit 1 Terms and Conditions.....13

List of Figures

Figure 1-1. Secure boot check flow6

Figure 1-2. dakey.h8

List of Tables

Table 1-1. Abbreviations.....4

1 Secure Boot

1.1 Overview

This document provides an overview of the secure boot feature in the MT8676 SoC and its accompanying SDK. During the design and deployment cycle of this product, functionalities and features may change. These changes will be documented in subsequent versions of this document. Additionally, this document describes how to enable secure features.

1.1.1 Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
DA	Download Agent download proxy
DAA	Download Agent Authentication
DRAM	Dynamic Random Access Memory
eFuse	Electronic Fuse
eMMC	Embedded MultiMedia Card
EVB	Evaluation Board
FIT	Flattened Image Tree
LK2	Litter Kernel 2
NVM	Non-Volatile Memory
PC	Power Control
ROM	Read-only Memory
SBC	Secure Boot Check
SDK	Software Development Kit
SoC	System-on-Chip
SRAM	Static Random Access Memory
TEE	Trusted Execution Environment
UART	Universal Asynchronous Receiver/Transmitter
USB	USB
USB DL	USB Download

1.1.2 BootRom

BROM (BootROM) is the software in SoC ROM that cannot be modified. It is the first software executed by the application processor. The main work summary is as follows:

- Perform basic hardware configuration for the System on Chip (SoC) as follows:
 - Set the Phase-Locked Loop (PLL).
 - Configure clocks.
 - Initiate system startup.

- Load the first-stage bootloader from the startup device UFS/eMMC/NAND to initiate the system.
- Communicate with the host PC tool to load Download Agent (DA) software for mirroring downloads to external Flash devices that lack software or have corrupted software.
- Perform secure boot checks on the first-stage bootloader or download agent using multiple secure boot check keys.

1.2 Secure Boot

1.2.1 Secure Boot Check Process

The Secure Boot check process is used to create a trusted execution chain from the hardware root of trust. It can be enabled by burning `SBC_EN` in eFuse. This process is performed every time the system recovers from power-on reset.

Figure 1-1 Demonstrates the basic secure boot check process. Detailed descriptions are as follows.

1. After power-on reset, BROM uses the following process to verify the secure boot check (SBC) public key (`SBC_PUBK`) in non-volatile memory (NVM, such as eMMC or NAND). BROM sequentially uses the public key hash (`SBC_PUBK_HASH`) from eFuse.
 - (1) Read the hash value of the secure boot check public key (`SBC_PUBK`) from eFuse (`SBC_PUBK_HASH`).
 - (2) Read the SBC public key (`SBC_PUBK`) from NVM.
 - (3) Calculate the hash value of the data from (2).
 - (4) Check whether the data from (1) and (3) is the same.
2. BROM loads and verifies `bl2.img` (Yocto LK2), which is the first stage of bootloader. First, BROM reads Yocto LK2 from NVM into the SoC's SRAM and uses the secure boot check public key (`SBC_PUBK`) to authenticate Yocto LK2. If Yocto LK2 verification is successful, Yocto LK2 will be executed. The verification method uses SHA256 to compute the hash value, RSA (2048 bits), and MTK or PSS padding (determined by the header of the first stage bootloader) for verification.

Note:

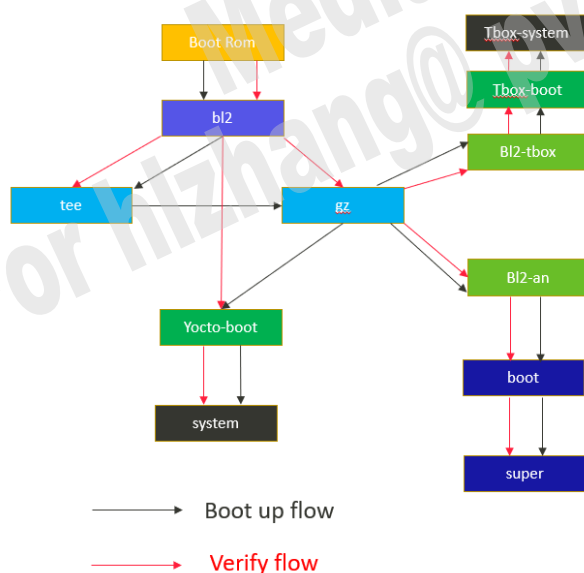
 - If the external storage is NAND flash memory, BROM supports the second copy of LK2. When BROM cannot load the first copy of LK2, BROM attempts to load and authenticate the second copy of LK2.
3. After the first stage bootloader, it runs in the Yocto LK2 stage, where LK2 will verify the Yocto FIT image, Hypervisor, and Android-related images.
4. Yocto LK2 loads the FIT image (such as the Yocto kernel and other FIT images) from NVM into a non-secure DRAM area and verifies it. It uses the VERIFIED public key embedded in LK2 to authenticate and verify the FIT signature of the FIT image, and detects the HASH values of each sub-image in the Fit image. The verification method uses SHA256 to compute the hash value, RSA (2048 bits), and MTK or PSS padding for verification.
5. Yocto LK2 also loads the MTK signed format image (such as TEE, Hypervisor image) from NVM into a secure DRAM area and verifies it. It uses the ROOT public key embedded in the LK2 program to authenticate the image's CERT1 and uses the IMAGE public key extracted from CERT1 to authenticate the image's CERT2. If CERT2 verification is successful, this image will be considered legitimate and will continue with the subsequent process. The verification method uses SHA256 to calculate the hash value, RSA (2048 bits), and MTK or PSS for verification.

6. After Yocto LK2 verifies the images required for 4 and 5 mentioned above, it will jump to TEE. TEE will start the Hypervisor image gz.img. The Hypervisor will first start the Yocto Kernel and then verify the startup of the Android LK2 image and Tbox LK2 image.
7. After the Yocto Kernel is executed, the root_check function provided by MediaTek in the Yocto Kernel can be used to implement sampling verification of the read-only system.
8. The method for verifying Android LK2 and Tbox LK2 by the Hypervisor needs to be provided by the Hypervisor vendor. Currently, it is Qianchuan.
9. Android LK2 will verify the Android Kernel through AVB. After executing in the Android Linux system, it can verify the read-only image through dm-verity (on block devices) in the Linux Kernel.

Note:

- Currently, dm-verity supports only block devices in Linux (eMMC on EXT4).

10. Tbox LK2 will load the Tbox Kernel image into the DRAM area for verification. The Tbox Kernel image is in FIT format. Tbox LK2 will use the VERIFIED public key embedded in Tbox LK2 to authenticate and verify the FIT signature of the FIT image and check the HASH values of each sub-image in the Fit image. The verification method uses SHA256 to calculate the hash value, RSA (2048 bits), and MTK or PSS padding for verification.
11. After the Tbox Kernel is executed, it can implement sampling verification of the read-only system through the root_check function provided by MediaTek in the Yocto Kernel.

Secure boot verify flow

MEDIATEK

MediaTek Proprietary and Confidential. © 2024 MediaTek Inc. All rights reserved.

key: rsa2048 + sha256**key name:** root key and image key**Verification method:** The hash of the SBC public key will be stored in efuse, and Boot Rom will use this key to verify Yocto LK2**key:** rsa2048 + sha256**key name:** Yocto image key**Verification method:** Yocto boot.img uses fit boot sign and verify methods. The public key will be stored in LK2 and will be verified when Yocto LK2 loads.**key:** rsa2048 + sha256**key name:** Yocto image key**Verification method:** Yocto OS uses mtd verify for verification, and the public key will be stored in Yocto boot.img.**key:** rsa2048 + sha256**key name:** Android/Tbox lk2 verify key**Verification method:** Android LK2 will use the rsa2048 + sha256 method to generate signature, and the public key will be stored in the hypervisor. The hypervisor will verify Android LK2 (requires the help of hypervisor owner goldenrivertek)**key:** rsa2048 + sha256**key name:** root key and image key**Verification method:** MTK Android secure boot sign + verify flow. Refer to the documentation: 《Secure 2.1 Configuration SOP.docx》. image from android build: tee.img, gz.img, mtk signature format modem.img.img..**key:** rsa2048 + sha256**key name:** DM verify key**Verification method:** Android's native DM verify flow, public key will be stored in Android boot.img**Figure 1-1. Secure boot check flow**

MEDIATEK

1.3 Download Agent Authentication (DAA)

When the device software in the external memory (eMMC/NAND/...) is empty or damaged, the host PC tool can communicate with the BROM in the SoC, load a DA software into the SRAM of the SoC and execute DA to download the image process. When Enable_DAA is burned, the DA will be authenticated by BROM, and the following process is called Download Agent Authentication (DAA).

1. The host PC tool sends the authentication file (AuthFile) to BROM via USB or UART.
2. BROM performs authentication on AuthFile.
 - (1) BROM reads the Secure Boot Check (SBC) public key (SBC_PUBK) from the AuthFile and uses the public key hash (SBC_PUBK_HASH~SBC_PUBK_HASH1) and the corresponding disable bits (SBC_PUBK_HASH_DIS~SBC_PUBK_HASH1_DIS) to authenticate the hash of the SBC key.
 - (2) BROM uses SBC_PUBK to verify AuthFile.
 - (3) BROM obtains the DAA key from the AuthFile.
3. The host PC tool sends DA.
4. BROM uses the DAA key to verify DA through SHA256/RSA-2048.
5. BROM jumps to DA to execute the firmware download process.

Note:

- The SBC_PUBK and DAA keys can be different to enhance security levels.
- In addition to BROM, the first-stage bootloader also supports the DA protocol, which can be used to verify DA using SHA256/RSA2048.

1.4 Security Feature Configuration

1.4.1 Generate Key Pairs.

Generate two pairs of keys (including private key and public key) in Yocto: SBC_KEY and VERIFIED_KEY. Place them in the Yocto system directory meta/meta-MediaTek/conf/machine/keys.

1. The command to generate the Yocto private key:

```
openssl genrsa -F4 -out sbc_key.pem 2048
openssl genrsa -F4 -out verified_key.pem 2048
openssl req -batch -new -x509 -key verified_key.pem -out verified_key.crt
```

2. The command to generate the Yocto public key:

```
openssl rsa -in sbc_key.pem -pubout > sbc_pubk.pem
openssl rsa -in verified_key.pem -pubout > verified_pubk.pem
```

Two pairs of keys are generated for android OS, including a private key and a public key: the root key (ROOT_KEY) and the image key (IMAGE_KEY). Use pem_to_der.py to convert the format of the root key to DER format in vendor/MediaTek/proprietary/scripts/sign-image_v2/der_extractor.

3. The command to generate an Android private key:

```
openssl genrsa -out root_prvk.pem 2048
python pem_to_der.py root_prvk.pem root_prvk.der
openssl genrsa -out img_prvk.pem 2048
```

4. The command to generate the Android public key:

```
openssl rsa -in root_prvk.pem -pubout > root_pubk.pem
python pem_to_der.py root_pubk.pem root_pubk.der
openssl rsa -in img_prvk.pem -pubout > img_pubk.pem
```

Keep using the same method to generate the DA key pair to sign and verify the DA (da_prvk.pem/da_pubk.pem). In addition, Yocto private key content is recommended overwrite the Android private key content, and the Yocto public key content overwrite the Android public key content, so that the configuration will be simpler.

1.4.1.1 Generate oemkey.h

Export the root key (root_pubk.der), use der_extractor to generate oemkey.h, this tool is located in the vendor/MediaTek/proprietary/scripts/sign-image_v2/der_extractor/ directory. Please place oemkey.h in the following path:

Android DA: \$DA_Kit/Raphael-da/custom/\$PLATFORM/oemkey.h

Yocto LK2: \$LK/target/\$PROJECT/inc/oemkey.h

Command:

```
chmod 777 der_extractor
./der_extractor root_pubk.der oemkey.h ANDROID_SBC
```

1.4.1.2 Generate dakey.h

The dakey.h file contains the public key for verifying DA_BR.bin by LK2. The corresponding private key is used to sign DA_BR.bin.

Command:

```
chmod 777 der_extractor
./der_extractor da_pubk.der dakey.h ANDROID_SBC
```

Place dakey.h in the following path:

Yocto LK2: \$LK2/target/\$PROJECT/include/dakey.h

Note:

- After generating dakey.h, please replace "OEM" with "DA" (Figure 1-2).

```
#ifndef _DAKEY_
#define _DAKEY_
#define DA_PUBK_SZ 256
#define DA_PUBK 0xD1, 0x64, 0x03, 0x46, 0x6C, 0x
0xE3, 0x32, 0xE1, 0x7D, 0xC0, 0
0x4E, 0xAA, 0xC2, 0xCB, 0x30, 0
0x68, 0x20, 0x0D, 0xE1, 0x7D, 0
0x73, 0x70, 0x50, 0xF0, 0x84, 0
0x0B, 0xB0, 0xCD, 0x60, 0x68, 0
0x71 0x5F 0x5B 0x5B 0x5B 0x5B
```

Figure 1-2. dakey.h

1.4.2 Enable Secure Boot.

1.4.2.1 Android Software Security Configuration

The default security configuration for the Android side software is enabled; you can check the following configurations:

1. LK2 configuration software (Hypervisor Android's LK2 is compiled in Yocto)

Yocto LK2: \$LK/project/\${PROJECT}.mk
MTK_SECURITY_SW_SUPPORT=yes

2. Kernel settings

32-bit Kernel

```
<kernel path>/arch/arm/configs/<project>_debug_defconfig
<kernel path>/arch/arm/configs/<project>_defconfig
```

64-bit Kernel

```
<kernel path>/arch/arm64/configs/<project>_debug_defconfig
<kernel path>/arch/arm64/configs/<project>_defconfig
```

```
CONFIG_MTK_SECURITY_SW_SUPPORT=y
```

1.4.2.2 Software Security Configuration for Yocto

```
meta/meta-mediatek-mt8xxx/conf/machine/[project].conf
SECURE_BOOT_ENABLE = "yes"
ENABLE_ROOTFS_CHECK= "yes"
```

1.4.2.3 TBox Software Security Configuration

```
meta/meta-mediatek-mt8xxx/conf/machine/[project]uos_tbox.conf
SECURE_BOOT_ENABLE = "yes"
ENABLE_ROOTFS_CHECK= "yes"
```

1.4.2.4 BROM Enables Secure Configuration.

The above three points all enable the security verification configuration at the software level. To enable the BROM verification first-loader (the first-loader in this project is Yocto LK2), you need to write the eFuse field.

1. Generate SCB_PUBK HASH:

Tool path: meta/meta-MediaTek/recipes-bsp/lk/files/pbp

Command:

```
chmod 777 der_extractor
python pbp.py -j sbc_key.pem -func keyhash_pss -o keyhash
```

2. Write SBC_PUBK HASH into eFuse:

Step 1: Use “hexdump -C keyhash” or “xxd -c 32 keyhash” to display the hexadecimal keyhash.

Note: You must use the -c parameter to generate a standard hexadecimal ASCII display result.

```
hexdump -C keyhash
00000000 16 b1 oe fc 5e 4e 06 76 e9 d9 6e 40 0c 51 ca 36 | .....
00000010 d1 be 93 d2 67 fd 3e af db f6 f7 89 4a 2c 40 18 |.....
xxd -c 32 keyhash
00000000 16b1 oefc 5e4e 0676 e9d9 6e40 0c51 ca36 d1be 93d2 67fd 3eaf dbf6 f789 4a2c
4018      .....
```

Do not use 'hexdump keyhash' for display.

```
hexdump keyhash
00000000 b116 fc0e 4e5e 7606 d9e9 406e 510c 36ca
00000010 bed1 d293 fd67 ad3e f6db 89f7 2c4a 1840
```

Step 2: Convert the result to a hexadecimal string. Remove the spaces between characters:

```
16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

Step 3: Execute the ewriter command:

```
ewriter 1 0 32 16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

Refer to the MTK_eFuse_Writer_User_Guide for more detailed usage of the ewriter tool.

Note:

- To enable the BROM secure boot verification function of the IC, in addition to writing the above SBC_PUBK0_HASH field, it is also necessary to write SBC_EN. The eFuse can only be written once, so it is not recommended to write eFuse to enable the BROM secure boot verification function of the IC during the project development phase.

1.4.3 Compilation Software

Compile the entire project.

1.4.4 Signature First-Loader

- Signing the first-loader (this project is Yocto LK2) will use two keys: sbc_key.pem and verified_key.pem.
- Key placement directory:
`<yocto branch>/meta/meta-mediatek/conf/machine/keys/`
- When compiling Yocto LK2, enabling the security configuration will automatically sign Yocto LK2.
- Signing process: Package the SBC public key and the verified public into the key cert, and signed them using the SBC private key; calculate the image hash, package the hash and verified pub key into the content cert, and sign them using the verified priv key; finally, concatenate the key cert and content cert to the end of the original image to generate the signed image.

1.4.5 Signature FIT Format Image

1. The first-loader signature (this project is Yocto LK2) requires the `verified_key.pem` key.
2. Key placement directory:
`<yocto branch>/meta/meta-mediatek/conf/machine/keys/`
3. When compiling each FIT image, enabling security configuration will automatically sign the corresponding image.
4. Signing process: The FIT image is an open-source standard format image, so the `uboot-mkimage` tool will be used to sign the image, mainly calculating the original HASH and storing it in the HASH node, and using the verified private key to sign the nodes in the FIT image except for the data node after concatenation, saving the signed result in the signature node.

1.4.6 Sign the Yocto File System Image.

1. The signed Yocto file system will also utilize the `verified_key.pem` key.
2. Key placement directory:
`<yocto branch>/meta/meta-mediatek/conf/machine/keys/`
3. When compiling each file system image, enabling security configuration automatically signs the corresponding image.
4. Signature process: MediaTek will provide the file system signature tool `mtb_verify`. This tool will extract N pieces of data from the original file system image based on the input parameters, each with a fixed size and fixed interval; it will concatenate the extracted parts to calculate the hash, use the verified private key to sign the HASH; and save the information of the input parameters and Signature at the end of the original image.

1.4.7 Sign the MTK Android Format Image.

1.4.7.1 Generate Keys for cert1 and cert2.

1. Use `root_prvk.pem` and `img_prvk.pem` to generate `cert1` and `cert2_key`.
2. Run the following command.

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/img_key_deploy.py mt6897
cert1_key_path=${KEY_PATH}/root_prvk.pem cert2_key_path=${KEY_PATH}/img_prvk.pem
root_key_padding=pss 2>&1 | tee SecureGen.log
```

Note:

- Do not enter the `./vendor/MediaTek/proprietary/scripts/sign-image_v2/` directory to execute `img_key_deploy.py`. Execute this command in the root directory of the code repository.
- Check whether `cert1` and `cert2_key` for all images have been updated in the following locations.
`vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert1\`
`vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert2_key\`

1.4.7.2 Signed Image

After generating `cert1` and `cert2_key`, run the signing script to generate `<image>-verified.bin` or `<image>-verified.img`.

Command:

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/SignFlow.py <platform> <project>
2>&1 | tee signflow.log
```

or

```
./vendor/mediatek/proprietary/scripts/sign-image/sign_image.sh 2>&1 | tee signflow.log
```

Lunch the project before compilation.

1.5 Signature DA

1. Key path setting

Export the DA public key (da_prvk.pem). Place it in the Android path

vendor/MediaTek/proprietary/scripts/secure_chip_tool/custom_keys.

2. Place the DA that needs to be signed in the `prebuilt/resignda/` directory, and execute the following command to sign the DA:

```
cd vendor/mediatek/proprietary/scripts/secure_chip_tool/
python MTK/resign_da.py prebuilt/resignda/DA_BR.bin MT6897
settings/Legacy/da/bbchips_pss.ini all out/resignda/DA_BR-resing.bin
```

1.6 Generate Authfile

The DA's public key is included in the authfile and is used by BROM to authenticate the DA. Therefore, if DAA is enabled, an authfile is required when using flashtool to download the image. All .ini files located in the following locations have been configured for this project.

./vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/Legacy/authfile/

Replace the .pem files for the DA and root private key located at the following location.

./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/

Authfile generation command:

```
python MTK/toolauth.py -i settings/Legacy/authfile/toolauth_key.ini -g
settings/Legacy/authfile/toolauth_gfh_config_pss.ini out/toolauth/auth_sv5.auth
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.